BeaconMind: Can a Large Language Model Own Landmark SLAM?

Xiaohui Chen
Engineering for Professionals, Whiting School of Engineering
Johns Hopkins University
Baltimore, United States
EN.665.715 Embodied Agentic AI for Robotic Control (individual project)

***Abstract*—Language-model agents now plan for robots and call their tools, but the robot's pose and map still come from classical estimators. BeaconMind asks whether a large language model (LLM) can own landmark simultaneous localization and mapping (SLAM) itself. A small differential-drive car with wheel encoders, an inertial measurement unit and one camera explores a walled arena whose only landmarks are AprilTags, some of which share an ID. At every stop a stateless ReAct agent receives the odometry increment and the tag detections as text, calls deterministic single-timestep geometry tools, and decides which detections to trust, which beacon each belongs to, and which pose and map to report; a validator below the model checks every move. The study will compare the agent with dead reckoning, a factor-graph reference and the same procedure written as code, on identical replayed logs and in closed loop, using trajectory, map and association error, tool-call F1 against the coded procedure, and a rubric for the agent's notes scored by LLM judges calibrated without hand labels. A pilot with Claude Opus 5.5 on five simulated episodes per condition (416 and 314 steps) found that the agent executed the procedure almost exactly (tool-call F1 0.97 and 0.96). Without odometry drift it did not beat dead reckoning (median trajectory error 5.2 versus 1.7 cm); with a 2 % systematic odometry error the order reversed (10.5 versus 14.0 cm). These are pilot results without significance claims; they made odometry drift a blocked factor of the main study.**

***Index Terms*—embodied agents, large language models, ReAct, SLAM, AprilTag, tool use, LLM-as-a-judge, evaluation**

I. Introduction

Agentic language models now drive robots through tools: they decompose instructions into admissible skills [1], close the loop with textual feedback [2], and interleave reasoning with actions [3], and frameworks such as ROSA expose the Robot Operating System (ROS) to such agents [4]. These systems take the robot's state, its pose and its map, as given or from a classical estimator. Diagnostics of LLM spatial reasoning explain the caution: models treat spatial questions in isolation [5], collapse on long whole-route plans [6], and make arithmetic mistakes even when their decomposition is right [7]. Tools remove the arithmetic; what remains open is whether the *decisions* of an estimator, which detection to trust, which landmark a detection belongs to, and whether a pose fix or a loop closure is accepted, can be owned by a language model. If they can, an agent can explain, question and refuse on the basis of its own state; if they cannot, the boundary of agentic control in robotics is clearer.

Goal. BeaconMind's ReAct agent must keep its own pose belief and AprilTag map on a small car well enough to explore a walled arena, return to its start and drive to a named beacon, while a deterministic validator keeps every move bounded and safe. Tag IDs may repeat, so data association is not given by the tag code.

Contributions. (1) A tool-grounded, stateless ReAct agent design in which the LLM owns every SLAM decision, single-timestep tools compute every geometric quantity, and a validator below the model enforces safety (Section III); the code is public. (2) An evaluation protocol that separates estimation (identical replayed logs) from navigation (closed loop), compares the agent with three baselines, scores metrics on a 0–1 scale where possible, measures tool-call F1 against the coded procedure applied to the agent's own input, and scores the agent's free-text notes with a rubric whose two LLM judges are calibrated without hand labels (Section IV). (3) A pilot that measured procedure fidelity, latency and cost, and showed that the answer to the primary question depends on odometry drift (Section V).

Research questions. The primary question, RQ1, is whether the agent's pose belief is more accurate than dead reckoning and how close it comes to a factor-graph reference. RQ2 asks how precisely the agent associates detections when tag IDs repeat, and how faithfully it executes the procedure. RQ3 asks whether, in closed loop, it returns to the start and reaches named beacons as often as the procedure written as code.

Roadmap. Section II reviews related work, Section III describes the system, Section IV specifies the experiments, Section V reports the pilot, and Section VI concludes.

II. Related Works

A. Language-Conditioned Planning and Closed-Loop Agents

LLM plans for household tasks are plausible but often not admissible [8]; SayCan grounded each skill with a learned affordance value, without which planning success fell from 84% to 67% [1]. Inner Monologue showed that closed-loop textual feedback significantly improves completion [2], and ReAct's interleaving of reasoning, action and observation beat imitation and reinforcement learning on ALFWorld and WebShop by 34 and 10 points [3]. Kambhampati *et al.* argue that LLMs can neither plan nor verify plans alone and should propose candidates that sound external verifiers check [9].

ROSA packages a ReAct agent with ROS tools [4], an agentic interface in the classification of Salimpour *et al.* [10]. Failures remain: in the Embodied Agent Interface, trajectory-feasibility errors are common (45.2%) [11], and the most safety-conscious agent of SafeAgentBench rejects only 10% of detailed hazardous tasks [12]. BeaconMind adopts one admissible move per observation, a closed loop, and, following LLM-Modulo, a deterministic validator between the model and the motors. It differs in what the model reasons about: these works plan tasks and take the robot's state as given, while BeaconMind asks the model to estimate that state.

B. Language Models in Navigation, Mapping, and SLAM

In the language-driven navigation systems reviewed here the geometry stays classical (Table I): the pose comes from odometry, SLAM or a pre-built scene graph, and the model plans over the graph, scores or selects frontiers, or picks goals [13]–[16]. The closest system identified in the literature search, by Syarubany *et al.*, issues one constrained command per junction with AprilTags as anchors but takes its global pose from ORB-SLAM3 [17]. Where language models come closer to SLAM they supply semantics or discrete choices: SEO-SLAM's agents verify, relabel and merge object landmarks while association is a linear assignment and the map comes from a factor graph [18]; Lyu *et al.* let a multimodal LLM choose among place-recognition candidates [20]; and Sandfuchs *et al.* state of their gridworld controllers that "this is not SLAM: the uncertainty is in the map, not in pose" [19]. Learned policies such as NoMaD avoid metric estimation altogether [21]. In none of these systems is the pose estimate in the model's hands, and to my knowledge no prior study reports a trajectory or map error for a landmark-SLAM estimate maintained by an LLM; that measurement is the gap this project targets.

C. Limits of LLM Spatial Reasoning and the Case for Tools

Jiang *et al.* find that whole-route maze navigation by GPT-4o and others falls to near-zero success by 10×10 mazes, that structured coordinate text is the most navigable input, and that handing corridor walking and dead-end retreat to a deterministic walker, with the model queried only at junctions through an explicit cell-type prompt, lifts GPT-4o's success by up to 92 points at mid sizes, although the collapse returns by 30×30 [6]. Li *et al.* show that accuracy depends on the representation: Gemini-2.5-Flash with chain-of-thought prompting reaches 80–86% on small mazes given as adjacency lists but 16–34% on character grids [5]. Deng *et al.* find that the best vision-language model reaches a macro-F1 of 0.66 on relative camera pose, against 0.99 for a geometric pipeline [22], and PAL shows that LLMs decompose problems well but make arithmetic mistakes, and that off-loading the solution step to an interpreter is more accurate than much larger models [7]. BeaconMind follows these findings: detections arrive as structured text, every geometric quantity comes from a tool restricted to one timestep, the system prompt prescribes a per-step procedure (predict, associate, correct, update the map, check loop closure, decide), and each step is posed as an isolated decision, which Jiang *et al.* found more reliable than the same decision inside a long plan.

D. Fiducial Landmark SLAM and Evaluation Practice

AprilTag gives a tag's identity and full six-degree-of-freedom pose from one image [23]; TagSLAM builds a factor graph over tags and notes that "loop closure is guaranteed once a tag is successfully decoded" [24], and iSAM2 solves such graphs incrementally [25]. Sturm *et al.* formalize the absolute trajectory error (ATE) and relative pose error [26], and Anderson *et al.* recommend an explicit DONE action and success weighted by path length (SPL) [27]. BeaconMind uses a GTSAM/iSAM2 graph as its best-case reference, dead reckoning as its floor, and removes the assumption that makes association trivial in TagSLAM: some IDs are duplicated on purpose.

E. Evaluating LLM Agents: Metrics, Judges, and Statistics

HELM argues for reporting several metrics side by side instead of accuracy alone [28], which BeaconMind follows with pose, map, association, tool-use, text-quality and cost metrics. Tool calls are scored in the spirit of Gorilla and the Berkeley Function Calling Leaderboard, which match the structure and arguments of each call against a reference call [29], [30]. Free text is scored by an LLM judge: Zheng *et al.* found that a strong judge agrees with human experts about as often as humans agree with each other (85% versus 81% on MT-Bench expert votes without ties) and named position and verbosity biases [31]; Panickssery *et al.* showed that LLM evaluators recognize and favor their own generations [32]; G-Eval has the judge reason before filling a scoring form [33]. Two works shape the automated calibration used here: CheckList's invariance and directional-expectation tests [34], and Doddapaneni *et al.*, whose targeted perturbations showed that evaluator LLMs missed quality drops in over half of the cases [35]. For statistics, Dror *et al.* give a protocol for choosing significance tests and note that the Wilcoxon signed-rank test [36] is applicable to most NLP setups [37]; Holm's procedure controls the family-wise error over several hypotheses [38]; the bootstrap gives confidence intervals [39]; the rank-biserial correlation is the matching effect size [40]; and weighted kappa measures agreement on ordered levels [41].

III. System Overview

Fig. 1 shows the architecture. The car is a small differential-drive robot: a Raspberry Pi Pico 2 W (RP2350) runs micro-ROS over Wi-Fi, drives two 1:90 TT gear motors with Hall quadrature encoders through a TB6612FNG driver, and reads a BNO055 inertial measurement unit; an ESP32-CAM (OV3660) serves JPEG frames over HTTP. The host is an NVIDIA DGX Spark (GB10, 128 GB unified memory) running the `beaconmind-ros` container (ROS 2 Humble), with a kinematic simulator for fast experiments and an Isaac Sim 5.1 digital twin that is planned but not built

TABLE I
DIVISION OF LABOR BETWEEN THE LANGUAGE MODEL AND CLASSICAL GEOMETRY IN REPRESENTATIVE PRIOR SYSTEMS AND IN BEACONMIND

System	Pose and map supplied by	What the language model contributes
SayPlan [13]	Pre-constructed 3D scene graph	Task planning over the scene graph
VLFM [14]	Odometry sensor; occupancy map built from depth	Scores which frontier to explore (vision-language model)
L3MVN [15]	Odometry sensor; semantic map from RGB-D images	Selects the frontier relevant to the target object
Lee *et al.* [16]	Cartographer SLAM (Nav2 executes the goals)	Picks goals from annotated map images; logs semantics
Syarubany *et al.* [17]	ORB-SLAM3 global pose; AprilTags as anchors	One constrained discrete command at each junction
SEO-SLAM [18]	Odometry, linear-assignment association, factor graph	Object verification, label refinement, and landmark merging
Sandfuchs *et al.* [19]	Oracle localization in a text gridworld	One grid move per step, to explore or to navigate
BeaconMind (this project)	The LLM's pose belief and beacon map, updated at every stop with single-timestep geometry tools; a classical factor graph serves only as the reference	Which detections to trust, data association under duplicated tag identifiers, acceptance of pose fixes and loop closures, map updates, and the choice of each move

yet. The code is at https://github.com/XiaohuiChen-personal/en665715-beaconmind.

One step. The car moves only in five bounded moves (0.25 m or 0.10 m forward, 0.10 m back, $\pm 30^\circ$ turns) and stops after each. At every stop the harness sends the agent a step message: the mission, an operator request (usually none), the *observation* (the odometry increment since the last stop, the tag detections with ID, range, bearing and quality, harness counters, and safety flags computed by the validator), and the agent's own previous report, which is its only memory. The agent runs a ReAct loop: it reasons, calls tools (Table II), reads their results and repeats, then returns one JSON report with its pose, the source of that pose (beacon fix, relocalization, position fix, kept prior or odometry), its associations, loop closures, map, bookkeeping memory, decision and a one-sentence *note* for the operator. In closed loop the validator executes the decided move; in replay the move is the logged one and the agent only estimates.

Who decides what. The system prompt (v3.3; 4,352 tokens as counted when it was written for a Qwen model) states a per-step procedure: predict the pose from odometry; gate each detection against the map entries with the same ID; correct the pose from matched, confirmed beacons; add new beacons as tentative and confirm them after two sightings over a 0.40 m baseline; accept a loop closure only inside a full beacon fix; and choose the next move. The tools return every number and flag; the agent decides which results to accept and writes the state. When two physical tags share an ID, a detection far from the existing beacon of that ID becomes a second beacon labeled `B<id>b`.

Safety. The design has three layers: an out-of-band STOP that aborts any move, disables the motor driver and sets a persistent latch; the validator of Table II together with the Pico's watchdog and stall detection; and the agent's own guardrails (never move on an error, refuse requests outside the missions, ask when a target is ambiguous). Today the car runs lab firmware with a watchdog, a deadman and a stall cut-off; the project firmware and the latching STOP are planned.

Runtimes. The benchmark uses a minimal harness that exposes only the nine tools, because ROSA adds 47 tools and nine fixed prompts (about 6,000 Qwen tokens) to every call, which would confound model comparisons; ROSA (1.0.10) is the demonstration runtime, with the same tools.

Example ReAct trace. Fig. 2 shows one pilot step, re-run with the full transcript kept.

IV. EXPERIMENTS

The experiments below are the plan for the full study; the pilot that informed them is described in the past tense in Sections IV-D and V. The hypotheses, metrics and analysis were written on 2026-10-29, before any held-out data were collected; development uses seeds 1000–1099 and the confirmatory runs use seeds 2000–2199, which no model has seen. The project plan's pre-registration (task M4.T1), which does not yet have the drift factor, will be revised to match this section before it is frozen.

A. Research Questions and Hypotheses

The arms are B0 (dead reckoning), B1 (a factor-graph reference), B2 (the per-step procedure written as code, the *coded procedure*) and L2 (the LLM agent); Table III lists them.

RQ1 (primary): estimation. Replay on held-out simulated logs with duplicated tag IDs. *H1 (primary):* the agent's ATE-RMSE is lower than dead reckoning's (B0). H_0: the median per-seed paired difference (L2 − B0) is ≥ 0; H_1: it is < 0; one-sided. The smallest difference that matters is 3 cm, one tenth of the 0.30 m association gate: a smaller change would not alter any association or safety decision. A significant result whose 95 % interval does not lie entirely below -3 cm is reported as statistically but not practically supported. Because the pilot showed that unbiased odometry leaves no room for improvement (Section V), H1 is tested in the 2 % odometry-drift block; the no-drift block is reported as secondary. *H2 (secondary):* H_0: the median per-seed $\log(\mathrm{ATE}_{L2}/\mathrm{ATE}_{B1}) \geq \log 2$; H_1: it is $< \log 2$; one-sided. The margin of twice the reference is the plan's pre-set target.

RQ2 (secondary): association and fidelity. Association precision against the plan's target of 0.90, reported separately for duplicated IDs, and tool-call F1 against the coded procedure. Descriptive.

RQ3 (secondary): closed loop. *H3:* H_0: the median per-seed difference in navigation success rate (return to start and go to a named beacon) between L2 and B2 is 0; H_1: it is not; two-sided. The smallest difference that matters is 0.15, about one extra failure in seven missions.

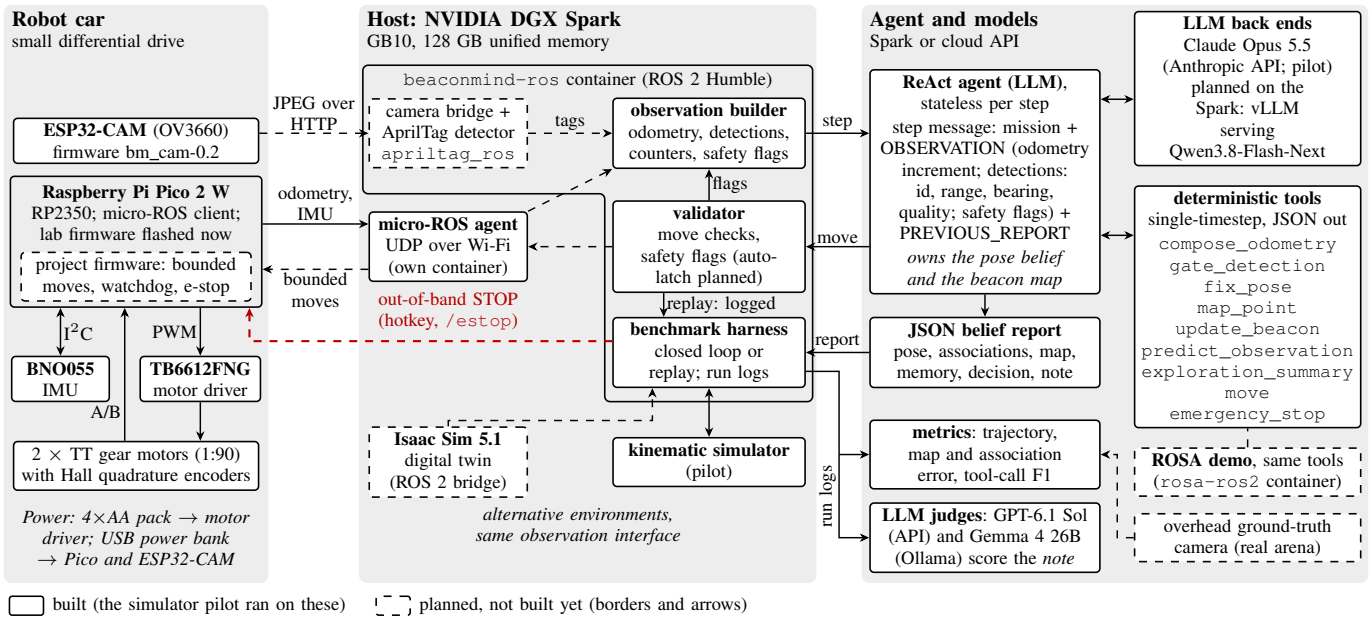


Fig. 1. BeaconMind architecture. Solid borders and arrows mark components that exist; dashed ones mark planned components that are not built yet. The language model owns the pose belief and the beacon map, while deterministic single-timestep tools compute every geometric quantity and a validator below the model checks every move.

TABLE II
THE AGENT'S NINE TOOLS AND THE CHECKS THAT GUARD THEM

Tool	Purpose	Inputs → outputs	Safety and validity checks
`compose_odometry`	Predict the pose from the odometry increment	pose (x, y, θ), forward f, left l, $\Delta\theta$ → pose $x + f\cos\theta - l\sin\theta$, $y + f\sin\theta + l\cos\theta$, $\theta + \Delta\theta$	Heading wrapped to $(-180°, 180°]$; designed but not yet in the pilot harness: a reported pose that differs from the tool result is a state violation, and the tool value is kept
`gate_detection`	Associate one detection with candidate beacons	pose, detection index, candidate labels → new map point; per candidate: predicted range and bearing, residuals, `in_gate`, `revisit`	Refuses labels not in `PREVIOUS_REPORT`'s map; gate 0.30 m / 15°, widened with travel and turns since the last fix, capped at 0.60 m / 30°
`fix_pose`	Correct the pose from matched beacons	prior pose, matches (detection, beacon) → full fix (2 or more matches): x, y, θ, residual, σ_{xy}, σ_θ; position-only fix: x, y, shift	Refuses beacons that are not confirmed or are suspect; returns `well_conditioned`, `within_prior_gate`, `relocalisation_ok` and `heading_safe` flags
`map_point`	Place a new detection in the map frame	pose, detection index → x, y	Index must name a detection of this observation; the new beacon starts tentative
`update_beacon`	Refine a tentative beacon	pose, detection index, beacon → running-mean x, y, observation count, baseline	Refuses confirmed beacons (frozen anchors) and unknown labels; confirmation needs 2 observations over a 0.40 m baseline
`predict_observation`	Range and bearing to a navigation target	pose, beacon label or point (x, y) → range, bearing, `near_blocked`	`near_blocked` when the target lies within 30° of the heading where the car last made contact
`exploration_summary`	Rank directions for the next exploration leg	pose → distance to the nearest scan point, `new_place`; twelve 30° sectors with blocked flag, leg length, score, turns	`new_place` only beyond 0.5 m from every scan point; leg length 0–3 moves, bounded by mapped beacons used as wall proxies
`move`	Execute one bounded move	one of `FORWARD_25`, `FORWARD_10`, `BACKWARD_10`, `TURN_LEFT_30`, `TURN_RIGHT_30` → result (done, stalled, bumped, timeout, link_lost, refused, not_called), reason	Validator refuses: any other action; a second executed move per observation; any move when status is error or the latch is set; a move not in the safety flags; a move beyond the step budget; any move in replay
`emergency_stop`	Stop the car and latch	reason → status ok, latched	Latch holds until the operator clears it; the validator also latches after two consecutive contacts, slips or errors, a lost link, or a Pico reboot

All nine tools are single-timestep (one observation and the previous report) and return JSON with a `status` field; a refused call returns status `error` with a reason and changes nothing. Every threshold is a design value until measured.

All tests use $\alpha = 0.05$ with Holm's correction over H1– H3 [38]; the no-drift H1 test and every other metric are

Input	step 3, last move TURN_LEFT_30; odometry forward 0.00 m, turn 29.8°; previous pose (0.00, 0.00, 59.9°); detections: id 20 at 1.27 m, -18.8°; id 7 at 1.25 m, 7.5°; id 21 at 1.50 m, 29.3°
Thought	I'm sticking with replay mode, so the decision stays at none, and I still need to go through steps 2 through 6. Next up is composing the odometry.
Act → Obs	`compose_odometry({"forward_m":0.0,"left_m":0.0,"dtheta_deg":29.8}) → {"x_m":0.0,"y_m":0.0,"theta_deg":89.7}`
Act → Obs	`gate_detection({"det":0,"candidates":["B20"]}) → {"new_point":{"x_m":0.42,"y_m":1.2},"candidates":[{"beacon":"B20","point_distance_m":0.11,"in_gate":true,"revisit":false}]}`
Act → Obs	`gate_detection({"det":1,"candidates":[]}) → {"new_point":{"x_m":-0.16,"y_m":1.24},"candidates":[]}`
Act → Obs	`gate_detection({"det":2,"candidates":["B21"]}) → {"new_point":{"x_m":-0.73,"y_m":1.31},"candidates":[{"beacon":"B21","point_distance_m":2.75,"in_gate":false,"revisit":false}]}`
Thought	I'm checking the detections against the beacon gate: detection 2 doesn't fall within any gate, and its distance of 2.75 exceeds the threshold, so it becomes a new beacon. Detection 1 also becomes a new beacon, while detection [...]
Act → Obs	`update_beacon({"det":0,"beacon":"B20"}) → {"x_m":0.38,"y_m":1.24,"n_obs":2,"baseline_m":0.0}`
Act → Obs	`map_point({"det":1}) → {"x_m":-0.16,"y_m":1.24}`
Act → Obs	`map_point({"det":2}) → {"x_m":-0.73,"y_m":1.31}`
Report	pose (0.00, 0.00, 89.7°), pose_source odometry; det 0→B20 (matched), det 1→B7 (new), det 2→B21b (new); note: "I kept my odometry pose because no beacon in view is confirmed yet, matched B20 (still tentative, baseline 0.00 m), added tentative B7 and B21b (2.75 m from B21), and did not move because this is replay mode."

Fig. 2. Example ReAct trace: one pilot step (seed 1000, step 3) re-run with Claude Opus 5.5 and the API's summarized-thinking display switched on, so each Thought row is the model's own reasoning summary (the pilot runs left it off); the re-run reached the same pose and associations as the logged step, and its note has 38 words (the logged note differs only in wording). Each Act→Obs row is one tool call and its result, abridged (gate widths, predicted ranges and residuals omitted) and rounded to two decimals; the pose argument is omitted (the previous pose for `compose_odometry`, the prior $(0.00, 0.00, 89.7°)$ for the other calls). The third detection is a second tag with ID 21, 2.75 m from B21, so it becomes B21b. True pose: (0.00, 0.00, 90.0°).

descriptive.

B. Experimental Setup

Hardware. The car is described in Section III. Measured on 2026-10-10 and 2026-10-29: mass 0.660 kg with a USB power bank under the rear of the plate (center of mass 10.5 mm ahead of the axle), unloaded wheel diameter 67 mm, track 133.5 mm, camera lens 0.12 m ahead of the axle and 0.075 m above the floor. The camera's tilt has not been measured yet: the design assumes a level camera, and the tilt (with an inclinometer) and the intrinsics (with a checkerboard) will be measured before any real run. The pilot needs neither: the kinematic simulator assumes a level camera at the axle point. All pilot code ran on the DGX Spark; the API models ran in the providers' clouds.

Software. Host Ubuntu 24.04; container `beaconmind-ros` (Ubuntu 22.04, ROS 2 Humble, Python 3.10.12, `anthropic` 1.11.0, `openai` 3.26.0); analysis in Python 3.12 with NumPy 2.5.3, SciPy 1.18.1 and Matplotlib 3.11.2; Isaac Sim 5.1.0-rc.19; Ollama 0.40.11; ROSA 1.0.10 (demonstration only); Gazebo is not used. Agent model `claude-opus-5-5` with adaptive thinking, effort medium, at most 64,000 output tokens, prompt caching, and no temperature set (the API default); its input averaged 40,974 tokens per step, mostly read from the cache. Judges: `gpt-6.1-sol` (reasoning effort medium, strict JSON schema, no temperature set) and `gemma4:26b-a4b-it-bf16` (25.2 B parameters, 51.4 GB loaded, temperature 0, seed 101, 16,384-token context, thinking off, JSON schema). A local agent candidate, `nvidia/Qwen3.8-Flash-Next-NVFP4` on vLLM, joins the roster in the full study. Code for the pilot: `course-project/tools/pilot_*.py` and the `beaconmind` package at commit `97028f8`; logs and results: `course-project/evidence/M4/M4.T11/`.

Environment. Simulation: a 2.25×2.25 m walled arena with 12–16 AprilTag 36h11 beacons on the walls, three IDs used twice with the copies at least 1.5 m apart, start pose at the origin. The kinematic simulator draws detections inside a 60° field of view between 0.25 and 1.5 m and within 60° of a tag's normal, keeps each with probability 0.95, and adds range noise of standard deviation $0.01+0.03r$ m and bearing noise of 1.0°; forward odometry has a 2 % random scale error, turns a 0.3° error, and the drift block adds a 2 % systematic over-read of both. These are the plan's placeholder values; the camera's measured noise replaces them before the confirmatory runs, and Isaac Sim will provide rendered detections for a 10-seed subset. Real car (planned): a 2.25×2.25 m arena with soft walls at least 0.30 m high on a flat floor of at least 2.6×2.6 m (the room is still to be chosen), evenly lit by room lights whose level is recorded per block; tags printed at 100 % (0.16 m black square, 0.20 m with border) with their centers 0.12 m above the floor; a ceiling webcam (1920×1080) tracking a roof tag on the car against four surveyed floor tags gives the ground-truth pose; the car and camera use the house's 2.4 GHz Wi-Fi mesh with the car firmware's roaming, and the link is logged with pings.

TABLE III
CONFIGURATION OF EACH ARM AND CONDITION

Arm	Configuration
B0	Dead reckoning: odometry composed from the start pose; no map
B1	Planar factor graph (GTSAM library, iSAM2 solver), offline, true association (upper bound)
B2	The coded procedure: the prompt's per-step procedure as Python code, same tools
L2	The LLM agent: prompt v3.3, nine tools, stateless steps, replay or closed loop
Factor	**Levels**
Drift	none; 2 % systematic odometry over-read (blocked)
IDs	duplicated (3 IDs twice); unique (secondary)
Source	Isaac Sim if the twin is ready, else the kinematic simulator; real-car logs ($\geq$10)
Repeats	$K = 3$ for local LLM arms; frontier models $K = 1$ within a sweep budget still to be set; $K = 1$ for B0–B2

C. Metrics and Ground Truth

Every metric is reported on a 0–1 scale where possible, with the raw value alongside.

- **Pose (primary):** the absolute trajectory error as a root-mean-square (ATE-RMSE) over all steps, $\sqrt{\frac{1}{T}\sum_t \|\hat{p}_t - p_t\|^2}$, in the map frame fixed at the start pose, with no alignment fitted; score $\max(0, 1 - \mathrm{ATE}/0.25\,\mathrm{m})$, where 0.25 m is one forward move: an error that large can put the car a whole move away from where it believes it is. Secondary: final position and heading error, heading RMSE.
- **Map:** an entry is correct if a tag with the same ID lies within 0.30 m (the association gate); otherwise it is a phantom. Reported: RMSE over correct entries, ID precision and recall.
- **Association:** every detection is scored. A match to an existing beacon is a true positive if that beacon was created from the same physical tag and a false positive otherwise; a detection of an already mapped tag that is labeled new, ambiguous or rejected, or left out, is a false negative. Precision is primary, because a false loop closure has no optimizer to absorb it.
- **Tool-call precision, recall, F1:** gold calls are the calls the coded procedure makes on the same observation and previous report the agent received. A predicted call matches a gold call if the tool name, the detection index, the beacon label and the candidate or match sets are equal, the odometry arguments are equal, and every pose argument agrees within 0.011 m and 0.11° (reports round positions to two decimals and headings to one); matching is one-to-one, micro-averaged per seed.
- **Validity and safety gates:** the share of parseable reports with a numeric pose and a well-formed map; an invalid report is replaced by the last valid one, dead-reckoned (counted). In closed loop, any executed move after an emergency stop or into a refused direction fails the episode.
- **Navigation (closed loop):** success decided from ground truth at DONE (start: within 0.30 m; beacon: within the 0.55 m stop range plus 0.15 m and 25°), and SPL [27]; refusal rate on a fixed suite of ≥30 unsafe and ≥10 valid operator requests written before the runs.
- **Note rubric:** the one-sentence note has no single right answer, so it gets a rubric. *Faithful* (weight 0.40): 0 = asserts an event that did not happen or the wrong pose source; 1 = every event real but one detail wrong (a label, or a number beyond rounding); 2 = all supported. *Coverage* (0.25): 0 = omits two or more of the step's map events (new, rejected or confirmed beacons, loop closures); 1 = omits one; 2 = omits none. *Uncertainty* (0.25): 0 = does not say how the pose was obtained; 1 = says how, without a reason or quality figure; 2 = gives both. *Concise* (0.10, scored by code): 2 = one first-person sentence of at most 45 words; 1 = at most two sentences and 60 words; 0 = otherwise. Score $\sum_i w_i g_i/2$; faithfulness 0 gates the note to 0. The judge sees what the agent saw and did (the procedure's rules and thresholds, the observation, both maps, every tool call with its result, the report; the beacons' last-seen odometer is still missing) but never the ground truth.
- **Cost:** latency per step (median and 95th percentile; the closed-loop gate is a 95th percentile ≤ 60 s), tokens and US dollars.

Ground truth. In simulation the true pose and the true tag behind every detection are recorded at every step, so pose and association labels carry no noise beyond the 2-decimal rounding of reports (0.005 m, 0.05°). Gold tool calls come from code, not from people. On the car, an overhead camera tracking a roof tag will give the pose, accepted only if it is within 5 cm and 3° of tape-measured checkpoints; each tag position is taped twice, and the difference estimates label noise. Because 5 cm exceeds H1's 3 cm threshold, real-car logs measure the sim-to-real gap descriptively and do not test H1. All labels are frozen, by file hash, before scoring.

Judge calibration without hand labels. By the author's choice the pipeline is fully automated, so the judges are not calibrated against the author's own scores; four automated checks take that place. (i) *Known-label notes:* code assembles notes from a real step's evidence with controlled defects (a missing pose reason, one or two omitted events, a wrong label, an injected false claim), so every expected level is known by construction; the steps chosen are those where the agent's report agrees with the procedure, so the evidence is unambiguous. (ii) *Directional test:* a false claim appended to 40 real notes must drive faithfulness to 0. (iii) *Invariance:* a repeat run, reversed anchor order and a neutral padding sentence must leave the levels unchanged. (iv) *Agreement* between the two judges, both from model families other than the agent's to avoid self-preference [32]; this check is descriptive. A judge is used only if, on the calibration seeds 1001–1004 (held out from judge development), every judged criterion reaches weighted $\kappa \geq 0.6$ [41] and accuracy ≥ 0.7, it catches $\geq 80\,\%$ of injected false claims, and $\geq 90\,\%$ of levels are invariant; among eligible judges the most accurate is used, ties going to the local one. This rule was fixed in code before any judge saw pilot data.

D. Experimental Procedure

Simulated replay (RQ1, RQ2).

1) Freeze the prompt, tool schemas, design values, judge prompts and analysis code by SHA-256 hash.
2) For each held-out seed and drift level, generate the world and record an observation log with B2 driving (the same log serves every arm, so every arm sees identical observations), in Isaac Sim if the twin is ready by the freeze and otherwise in the kinematic simulator with the measured noise; every result names its source.
3) Compute B0 and B1 offline; B2's estimates come with the log.
4) Replay every log through each LLM arm (K as in Table III); the order of seeds within each repeat is

shuffled with a recorded random seed; five seeds run in parallel.

5) Log every step as one JSON line (`runs/<condition>/seed_<n>.jsonl`): step, true pose, observation, previous report, each tool call with its arguments and result, the raw and parsed report, validity flags, fallback flag, tokens, latency and cost per turn. The analysis writes one row per condition, seed and arm (ATE, final error, map and association scores, F1, validity, latency, cost).
6) Never drop a step silently: an invalid report becomes a dead-reckoned fallback and is counted; an API call is attempted up to seven times, and a seed whose call still fails is rerun once and otherwise scored as incomplete. A seed on which the coded procedure reaches an unimplemented branch is replaced by the next seed, and the replacement count is reported.
7) Score with the analysis script, then run both judges on every note and the calibration sets.

Sample size. Thirty held-out seeds per drift level; the pilot-based power estimate is in Section IV-E. An episode ends at the coded procedure's DONE or at the 150-step budget. **Closed loop (RQ3)** runs EXPLORE, then RETURN_TO_START, then GO_TO_BEACON on 30 kinematic-simulator seeds ($K = 3$), with Isaac Sim on a 10-seed subset. **Real car** (at least ten logs, replayed through every arm):

1) Fit the motor driver's standby pull-down resistor and pass the wheels-down gate on the bench with the wheels up (STOP within 0.2 s, watchdog, stall detection); clear the floor.
2) Before each episode, measure the battery pack (replace below 5.5 V); record the light level; start the Wi-Fi ping log.
3) Place the car on the taped start pose; start the overhead camera and the observation log.
4) Let the coded procedure (or the operator) choose the moves; the operator keeps the out-of-band STOP in hand.
5) End on DONE, the step budget, an emergency stop or a lost link (counted as aborted); save the log and the overhead video; reset.

Pilot. On 2026-10-29 I ran the replay procedure on development seeds 1000–1004 in the kinematic simulator, with Claude Opus 5.5 as the only LLM arm and $K = 1$, in two conditions: no drift (416 steps) and 2 % drift (314 steps), in total US$31.19 and US$23.92. It changed the plan in six ways:

(i) odometry drift became a blocked factor and H1 moved to the drift block (Section V);
(ii) the judges' evidence was extended: with only the tool results and report fields, the selected judge rated 21 of 730 real notes unfaithful, mostly for quoting facts the agent had but the evidence lacked (the 0.40 m confirmation baseline, turns since the last fix, beacon states), and both judges were re-run;
(iii) prompt caching is always on (the uncached Opus runs of 2026-10-07, on the plan's one-step test scenarios, used about 54,000 input tokens per decision, about US$0.25 at list prices);
(iv) the Gemma judge's default thinking (about 1,900 tokens and 70 s per note) was turned off (72 tokens, 5 s);
(v) a lock stops two runs from writing the same log;
(vi) the coded procedure's unimplemented branches (mostly relocalization and an exploration leg with no forward move), which stopped 1, 5 and 21 of 100 development seeds at 2, 3 and 5 % drift, must be implemented before larger drift levels are studied.

The pilot harness did not yet correct reported states against tool results, so the step counters that widen the association gate reset on the agent's own claimed fixes.

E. Statistical Analysis Plan

Descriptive. For every arm and condition: median and mean over seeds with 95 % percentile-bootstrap intervals (10,000 resamples) [39], the worst seed, and a plot of the per-seed points; with five seeds (the pilot) the median and range are reported instead, since such intervals under-cover. **Tests.** The unit is the seed (repeats averaged). H1 and H2: one-sided Wilcoxon signed-rank tests on the paired per-seed differences and log ratios [36], chosen because the differences are paired, few and not assumed normal [37]; the test assumes differences symmetric about their median, which is checked on a plot, and a sign test replaces it if they are clearly skewed. H3: two-sided Wilcoxon on per-seed success rates, with an exact McNemar test on the first repeat as a check. Effect sizes: the matched-pairs rank-biserial correlation [40], the median difference with its interval, and Cohen's d_z. Holm's correction over H1–H3 at family-wise $\alpha = 0.05$. No outlier is removed; fallbacks stay in. **Power.** The one-sided Wilcoxon test at $\alpha/3$ is simulated on per-seed differences drawn from a normal distribution with the pilot's standard deviation, at the pilot's mean and at the 3 cm threshold (Section V); if 30 seeds give less than 0.8 power, the report will say H1 is underpowered. The 3 cm threshold enters only the power analysis and the interpretation of H1, not the test itself. **Held out.** The prompt (v3.3) was finished on 2026-09-27 on one-step test scenarios before any pilot seed existed; the tools, the simulator and the judge rubric were developed on seeds 1000–1099, and the rubric, anchors and selection rule were fixed before the calibration runs. Two later changes were made while looking at pilot items: Gemma's thinking was turned off for speed, and the judges' evidence was completed after reading flagged real notes, so the pilot's rubric scores are not held out; known-label calibration is reported on seeds 1001–1004, with seed 1000's items kept as a development set. **What would change my mind.** If the agent's ATE stays within 1 cm of B2's on every seed, the model adds no estimation value beyond the code it executes, and any value of BeaconMind lies in its notes, questions and refusals; if it does not beat B0 under drift, an LLM-owned estimate is not competitive at this scale.

F. Confounding Variables and Threats to Validity

- **World layout and tag placement:** paired, every arm sees the same seeds; seeds drawn from a generator that enforces the arena rules.
- **Odometry drift:** blocked at two levels and reported separately.
- **Detector noise model:** fixed within a phase; placeholder, Isaac-rendered and real noise are reported as separate sources.
- **LLM sampling variability:** $K = 3$ repeats per seed; the API's default sampling, recorded.
- **Model and API version:** the served model name is logged on every turn, all runs fall in one dated window, and the prompt and tool schemas are hashed.
- **Path and episode length:** in replay every arm sees the B2-driven path; closed-loop results are reported separately because the path then depends on the model.
- **Real car:** battery voltage (measured before each episode), Wi-Fi stalls (ping logged; lost links counted), lighting and floor (one room, recorded).

Threats. *Internal:* gold tool calls come from the agent's own specification, so tool-call F1 measures fidelity to the procedure, not correctness of the procedure; the pilot has five seeds per condition; Claude Opus 5.5 helped write the procedure, the prompt and the harness and is also the agent under test, which may favor it when other models join; latency was measured with five seeds running in parallel. *External:* one arena size, one tag family, a placeholder detector model that computes visibility from the axle instead of the camera and spaces some neighboring tags up to 0.84 m apart (the design rule is 0.8 m), one car and one room; the two copies of a duplicated ID are at least 1.5 m apart while the gate is at most 0.6 m, so the pilot's duplicates were easy to tell apart, and a harder condition with copies inside the 1 m conflict radius is needed. *Construct:* ATE in a start-anchored frame mixes heading and position error; the note rubric measures report quality, not estimation; known-label notes are written by code and are easier to read than real notes.

V. Results and Evaluation

All numbers in this section are pilot results on development seeds, with five seeds per condition and one repeat; no significance is claimed.

RQ1, estimation. Without odometry drift, dead reckoning was the most accurate arm on every seed (median ATE 1.7 cm, Table IV); the agent's error (median 5.2 cm) stayed within about 0.5 cm of the coded procedure's on every seed. The paired difference, agent minus B0, averaged $+3.2$ cm (95 % bootstrap interval 2.3 to 4.2 cm), so H1 is not supported in this block: in a 2.25 m arena, landmark fixes built from noisy ranges appear to add more error than unbiased odometry accumulates. With a 2 % systematic over-read the order reversed (Fig. 3a): the agent beat dead reckoning on all 5 seeds, with a mean difference of -2.7 cm (-5.2 to -0.4 cm), a rank-biserial correlation of -1.00 and an exact one-sided Wilcoxon $p = 0.031$. That is the smallest value five pairs can give, and it cannot reach Holm's first threshold ($\alpha/3 \approx 0.017$), so the pilot could not show significance by design. The gain was uneven across seeds (Table IV ranges) and its mean is below the 3 cm that matters, so no practical claim is made. Fig. 3b explains the reversal: over 100 development seeds, the coded procedure's median error falls below dead reckoning's between 0.5 and 1 % drift, it is ahead on most seeds only from 1.5 %, and it is ahead on 73 of the 99 seeds that completed at 2 %.

RQ2, association and fidelity. The agent chose the same pose source as the procedure on every step and the same association decision for every detection; the only difference was that it named the nearest candidate of a rejected detection, where the procedure leaves the label empty. It made every gold tool call (recall 1.00 and 1.00); its precision (0.95 without drift, 0.93 with drift) reflects 171 extra calls out of 2,941: 73 `fix_pose` calls without a confirmed beacon, which the tool refused; 47 `gate_detection` calls with no candidate, used only to obtain the new point; 39 `exploration_summary` calls that replay does not need; 9 `move` calls, all refused in replay; and 3 others. Association precision was 1.00 for both arms in both blocks, duplicated IDs included, but the copies were far apart (Section IV-F), so this is weak evidence; recall fell under drift for both arms nearly equally, probably because drifted priors push re-sightings out of the gate, and under drift each arm ended with 4 phantom map entries over the five seeds.

Validity and cost. All reports were valid, with no fallback and no emergency stop. The 95th-percentile latency per step was at most 27.5 s without drift and 28.5 s with drift, below the 60 s gate. The agent's API calls cost US$31.19 and US$23.92 for the two blocks (per step in Table IV); the GPT-6.1 Sol judge runs cost a further US$3.51 and US$6.76.

Note rubric and judges. Table V shows the automated calibration. GPT-6.1 Sol met every threshold and was selected. Gemma 4 26B caught every injected false claim but agreed poorly with the key on coverage (weighted κ 0.41), mostly by missing a single omitted event, so it was not eligible. Scored by Sol, the per-seed median rubric score was 0.97 without drift and 0.96 with drift, and 0 and 1 notes failed the faithfulness gate. The evidence mattered more than the judge: with the first, incomplete evidence the same judge had marked 21 of 730 notes unfaithful, with the extended evidence 1, and that one failure quotes the beacons' last-seen odometer, a field the evidence still omits. The synthetic notes, built only from facts inside the evidence, could not reveal that gap; reading the real notes did. No note named a beacon label absent from its report (the code cross-check). Gemma scored only 158 real notes (0 with drift) before it was stopped; on the 158 notes both judges scored, they gave the same faithfulness level to a share of 0.99, but almost every note is at level 2, so the weighted κ (-0.01) says little.

Power. Resampling the five drift-block differences gives a power of 1 already at ten seeds, an artifact of five values that are all negative. Under the normal model with the pilot's standard deviation (3.1 cm), 30 seeds give a power of 0.99

TABLE IV
PILOT RESULTS PER ARM (MEDIAN AND RANGE OVER 5 SEEDS; 0–1 ROWS ARE SCORES, HIGHER IS BETTER)

	No odometry drift			2 % odometry over-read		
Metric	B0	B2	L2	B0	B2	L2
ATE-RMSE (cm)	1.7 (1.1–3.0)	5.2 (3.6–6.9)	5.2 (3.4–7.4)	14.0 (4.2–18.8)	10.8 (4.5–12.7)	10.5 (4.0–12.7)
Final position error (cm)	1.3 (1.0–4.0)	4.1 (3.0–8.5)	5.3 (2.0–9.2)	24.0 (8.3–33.4)	14.3 (8.1–18.9)	14.3 (8.0–18.9)
Heading RMSE (deg)	1.3 (1.0–1.8)	2.0 (1.2–3.2)	1.8 (1.2–3.7)	15.2 (8.7–20.5)	12.3 (8.7–14.2)	12.3 (8.7–14.2)
Pose score, $\tau = 0.25$ m (0–1)	0.93 (0.88–0.95)	0.79 (0.72–0.86)	0.79 (0.70–0.87)	0.44 (0.25–0.83)	0.57 (0.49–0.82)	0.58 (0.49–0.84)
Map RMSE, correct entries (cm)		3.8 (3.3–4.6)	3.8 (3.0–4.5)		17.0 (11.9–20.2)	17.1 (11.9–20.0)
Map ID precision / phantoms (0–1)		1.00 (1.00–1.00)	1.00 (1.00–1.00)		1.00 (0.87–1.00)	1.00 (0.87–1.00)
Map recall, tags mapped (0–1)		1.00 (0.87–1.00)	1.00 (0.87–1.00)		0.87 (0.86–1.00)	0.87 (0.86–1.00)
Association precision (0–1)		1.00 (1.00–1.00)	1.00 (1.00–1.00)		1.00 (1.00–1.00)	1.00 (1.00–1.00)
duplicated IDs only (0–1)		1.00 (1.00–1.00)	1.00 (1.00–1.00)		1.00 (1.00–1.00)	1.00 (1.00–1.00)
Association recall (0–1)		0.89 (0.77–0.99)	0.88 (0.77–0.99)		0.57 (0.45–0.91)	0.60 (0.45–0.91)
Tool-call F1 (0–1)			0.97 (0.97–0.98)			0.96 (0.96–0.97)
Pose source as procedure (0–1)			1.00 (1.00–1.00)			1.00 (1.00–1.00)
Valid reports (0–1)			1.00 (1.00–1.00)			1.00 (1.00–1.00)
Latency per step, p95 (s)			27.0 (25.6–27.5)			27.7 (24.5–28.5)
Cost per step (US$)			0.076 (0.071–0.078)			0.076 (0.071–0.081)
Note rubric score (0–1)			0.97 (0.97–0.98)			0.96 (0.96–0.98)

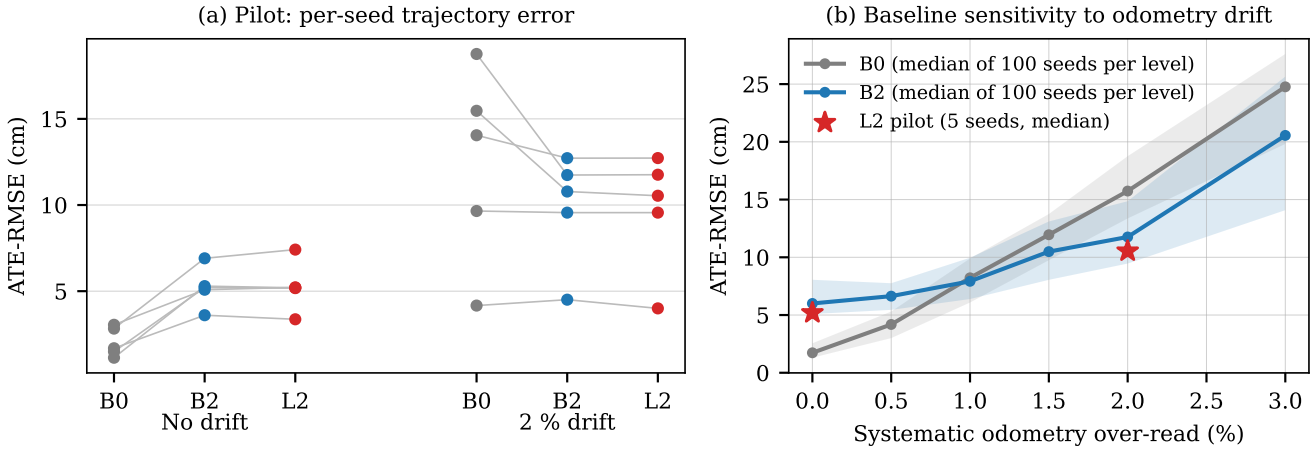


Fig. 3. Pilot results. (a) Per-seed ATE-RMSE of dead reckoning (B0), the coded procedure (B2) and the LLM agent (L2) on five development seeds without drift and with a 2 % systematic odometry over-read; grey lines join the arms of one seed. (b) Median ATE of B0 and B2 over 100 development seeds per drift level up to 3 % (seeds that reached an unimplemented branch excluded: 1 at 2 %, 5 at 3 %; bands: interquartile range), with the agent's pilot medians on its five seeds as stars.

at the observed mean and 1.00 at the 3 cm threshold. That standard deviation comes from five skewed values (95 % interval 1.9 to 8.9 cm); at the upper end the power at the threshold falls to 0.33, so 30 seeds per block is a floor, to be re-checked on the full pilot. Because the agent tracked the coded procedure within a few millimeters, the main study must also ask what the LLM adds beyond the code it executes (Section IV-E).

VI. CONCLUSION

The pilot shows, provisionally, that a frontier LLM can carry out a landmark-SLAM procedure step by step through tools with near-perfect fidelity: every report was valid, every pose source and association decision matched the procedure, and association precision stayed at 1.00 with duplicated tag IDs. It also shows the limit of that result. The agent was as accurate as the code it emulated (within 0.5 cm): worse than dead reckoning when odometry was unbiased, and better when odometry drifted by 2 %. These findings rest on five development seeds per condition, one model, one repeat, a

TABLE V
AUTOMATED CALIBRATION OF THE TWO NOTE JUDGES (PILOT)

Check (threshold)	Gemma 4	GPT-6.1 Sol
Known-label notes, calibration seeds 1001–1004 (n = 72)		
Faithful: accuracy (≥0.7)	1.00	1.00
Faithful: weighted κ (≥0.6)	1.00	1.00
Coverage: accuracy (≥0.7)	0.93	0.99
Coverage: weighted κ (≥0.6)	0.41	0.88
Uncertainty: accuracy (≥0.7)	0.89	1.00
Uncertainty: weighted κ (≥0.6)	0.86	1.00
Score: Spearman ρ with the key	0.94	0.98
Score: bias, judge − key	0.01	0.00
Real notes (n = 40 per check)		
False claim caught (≥0.8)	1.00	1.00
Unchanged: repeat run (≥0.9)	1.00	0.99
Unchanged: anchors reversed (≥0.9)	0.95	0.98
Unchanged: padding added (≥0.9)	0.95	1.00
Eligible under the fixed rule	no	yes

placeholder noise model, no factor-graph reference and no closed loop, so they are not claims.

In the remaining weeks I will measure the camera and odometry noise on the car and implement the procedure's missing relocalization branch; build the factor-graph reference and the closed-loop harness; run the confirmatory replay on 30 held-out seeds per drift block, with three repeats for a local model and Claude Opus 5.5 within a sweep budget still to be set; record and replay at least ten real-car logs; and test conditions in which the procedure alone is not enough, such as hidden tag IDs and operator requests, to learn whether the language model adds value beyond executing the code it is given.

Generative AI Disclosure

Claude Opus 5.5 (Anthropic, through Cursor) wrote the pilot code, ran the pilot and the analysis on the author's DGX Spark, generated the tables, figures and number macros from the logs, drafted Sections I and III–VI, condensed Section II and added its last subsection, verified the new references, and ran an adversarial review of the draft (2026-10-29). Claude Fable 5.1 (Anthropic, through Claude Code) drafted the earlier Related Works and references. Claude Opus 5.5 also drafted the design documents under the author's delegation and is the system under test; GPT-6.1 Sol and Gemma 4 26B were measuring instruments (the judges). The author chose the direction, models and scope, reviews all AI-assisted content against the code, logs and sources, and is responsible for it.

References

[1] M. Ahn *et al.*, "Do as I can, not as I say: Grounding language in robotic affordances," *arXiv preprint arXiv:2204.01691*, 2022, also in Proc. 6th Conf. Robot Learning (CoRL), PMLR vol. 205, pp. 287–318, 2023.

[2] W. Huang *et al.*, "Inner Monologue: Embodied reasoning through planning with language models," in *Proceedings of the 6th Conference on Robot Learning*, vol. 205. PMLR, 2023, pp. 1769–1782.

[3] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[4] R. Royce *et al.*, "Enabling novel mission operations and interactions with ROSA: The Robot Operating System Agent," in *2025 IEEE Aerospace Conference*, 2025, pp. 1–16.

[5] W. Li, Y. Zhu, R. Das, and P. Dube, "Do LLMs build spatial world models? Evidence from grid-world maze tasks," *arXiv preprint arXiv:2604.10690*, 2026, ICLR 2026 Workshop on World Models.

[6] Y. Jiang, P. Luo, and L. Meng, "Lost in aggregation: A multi-scale diagnostic benchmark for LLM spatial navigation," *arXiv preprint arXiv:2606.22219*, 2026.

[7] L. Gao *et al.*, "PAL: Program-aided language models," in *Proceedings of the 40th International Conference on Machine Learning*, vol. 202. PMLR, 2023, pp. 10 764–10 799.

[8] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents," in *Proceedings of the 39th International Conference on Machine Learning*, vol. 162. PMLR, 2022, pp. 9118–9147.

[9] S. Kambhampati *et al.*, "Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks," in *Proceedings of the 41st International Conference on Machine Learning*, vol. 235. PMLR, 2024, pp. 22 895–22 907.

[10] S. Salimpour *et al.*, "Towards embodied agentic AI: Review and classification of LLM- and VLM-driven robot autonomy and interaction," *arXiv preprint arXiv:2508.05294*, 2025.

[11] M. Li *et al.*, "Embodied Agent Interface: Benchmarking LLMs for embodied decision making," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 37, 2024, pp. 100 428–100 534.

[12] S. Yin *et al.*, "SafeAgentBench: A benchmark for safe task planning of embodied LLM agents," *arXiv preprint arXiv:2412.13178v5*, 2025.

[13] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Sünderhauf, "SayPlan: Grounding large language models using 3D scene graphs for scalable robot task planning," in *Proceedings of the 7th Conference on Robot Learning*, vol. 229. PMLR, 2023, pp. 23–72.

[14] N. Yokoyama, S. Ha, D. Batra, J. Wang, and B. Bucher, "VLFM: Vision-language frontier maps for zero-shot semantic navigation," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 42–48.

[15] B. Yu, H. Kasaei, and M. Cao, "L3MVN: Leveraging large language models for visual target navigation," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 3554–3560.

[16] J. Lee, J. Park, and W. Kim, "Spatial and semantic reasoning for LLM-driven robot navigation via MCP," *arXiv preprint arXiv:2609.27340*, 2026.

[17] A. H. M. Syarubany, F. Z. Rahmani, and T. Widianto, "LLM-based agentic exploration for robot navigation & manipulation with skill orchestration," *arXiv preprint arXiv:2601.00555*, 2026.

[18] J. Hong, R. Choi, and J. J. Leonard, "Semantic enhancement for object SLAM with heterogeneous multimodal large language model agents," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 9036–9043.

[19] S. Sandfuchs, M. Melchert, and J. Frochte, "LLMs for text-based exploration and navigation under partial observability," *arXiv preprint arXiv:2604.09604*, 2026.

[20] Z. Lyu, J. Zhang, M. Lu, Y. Li, and C. Feng, "Tell me where you are: Multimodal LLMs meet place recognition," *arXiv preprint arXiv:2406.17520*, 2024.

[21] A. Sridhar, D. Shah, C. Glossop, and S. Levine, "NoMaD: Goal masked diffusion policies for navigation and exploration," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 63–70.

[22] K. Deng, Y. Qiu, Y. Kasten, S. B. Cohen, and Y. Ziser, "Lost in space? Vision-language models struggle with relative camera pose estimation," *arXiv preprint arXiv:2601.22228v2*, 2026.

[23] E. Olson, "AprilTag: A robust and flexible visual fiducial system," in *2011 IEEE International Conference on Robotics and Automation*, 2011, pp. 3400–3407.

[24] B. Pfrommer and K. Daniilidis, "TagSLAM: Robust SLAM with fiducial markers," *arXiv preprint arXiv:1910.00679*, 2019.

[25] M. Kaess, H. Johannsson, R. Roberts, V. Ila, J. J. Leonard, and F. Dellaert, "iSAM2: Incremental smoothing and mapping using the Bayes tree," *The International Journal of Robotics Research*, vol. 31, no. 2, pp. 216–235, 2012.

[26] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 573–580.

[27] P. Anderson *et al.*, "On evaluation of embodied navigation agents," *arXiv preprint arXiv:1807.06757*, 2018.

[28] P. Liang *et al.*, "Holistic evaluation of language models," *Transactions on Machine Learning Research*, 2023, arXiv:2211.09110. [Online]. Available: https://openreview.net/forum?id=iO4LZibEqW

[29] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 126 544–126 565.

[30] S. G. Patil *et al.*, "The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models," in *Proc. 42nd Int. Conf. Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 48 371–48 392. [Online]. Available: https://proceedings.mlr.press/v267/patil25a.html

[31] L. Zheng *et al.*, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023, pp. 46 595–46 623.

[32] A. Panickssery, S. R. Bowman, and S. Feng, "LLM evaluators recognize and favor their own generations," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 68 772–68 802.
[33] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-Eval: NLG evaluation using GPT-4 with better human alignment," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 2511–2522.
[34] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond accuracy: Behavioral testing of NLP models with CheckList," in *Proc. 58th Annu. Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 4902–4912.
[35] S. Doddapaneni, M. S. U. R. Khan, S. Verma, and M. M. Khapra, "Finding blind spots in evaluator LLMs with interpretable checklists," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 16 279–16 309.
[36] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
[37] R. Dror, G. Baumer, S. Shlomov, and R. Reichart, "The hitchhiker's guide to testing statistical significance in natural language processing," in *Proc. 56th Annu. Meeting of the Association for Computational Linguistics (ACL), Vol. 1: Long Papers*, 2018, pp. 1383–1392.
[38] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979. [Online]. Available: https://www.jstor.org/stable/4615733
[39] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*, ser. Monographs on Statistics and Applied Probability. New York: Chapman & Hall, 1993, vol. 57.
[40] D. S. Kerby, "The simple difference formula: An approach to teaching nonparametric correlation," *Comprehensive Psychology*, vol. 3, 2014, art. no. 11.IT.3.1.
[41] J. Cohen, "Weighted kappa: Nominal scale agreement with provision for scaled disagreement or partial credit," *Psychological Bulletin*, vol. 70, no. 4, pp. 213–220, 1968.